

Roadmap: Desarrollador Python para Sistemas Web

Una guía orientada a acciones y etapas de construcción, no solo dominio técnico del lenguaje.

1. 🌀 PLANIFICACIÓN E IDEACIÓN

Antes de escribir código

- **Análisis de requisitos**
 - Entender qué problema resuelve el sistema
 - Identificar usuarios finales y sus necesidades
 - Definir alcance MVP (Minimum Viable Product)
 - **Arquitectura conceptual**
 - Diagrama de componentes principales
 - Flujo de datos entre servicios
 - Identificar dependencias externas
 - **Stack tecnológico**
 - Elegir framework web (Django, FastAPI, Flask)
 - Decidir base de datos (PostgreSQL, MongoDB, etc.)
 - Evaluar integraciones de terceros (APIs, servicios)
 - Planificar infraestructura (local, cloud, on-premise)
 - **Estimación y timeline**
 - Desglosar en sprints o milestones
 - Identificar riesgos técnicos temprano
-

2. ⚙️ CONFIGURACIÓN INICIAL DEL PROYECTO

Estructurar para el éxito desde el inicio

- **Gestión de entornos**
 - `venv` o `conda` para aislar dependencias
 - Archivos `.env` para secretos y configuración
 - Diferenciación: dev, staging, producción
- **Gestión de dependencias**
 - `requirements.txt` o `pyproject.toml`

- Versionado de librerías
 - Auditoría de vulnerabilidades (pip-audit, safety)
 - **Control de versiones**
 - Estrategia de branching (Git Flow, trunk-based)
 - `.gitignore` apropiado
 - Convenciones de commit
 - **Estructura de carpetas**
 - `src/`, `tests/`, `docs/`, `config/`
 - Separación clara de responsabilidades
 - Empaquetación modular
 - **Herramientas de desarrollo**
 - Linting (Black, Ruff, Pylint)
 - Type checking (mypy, pyright)
 - Pre-commit hooks para calidad automática
-

3. 🏗️ CONSTRUCCIÓN DEL BACKEND

Crear la lógica central del sistema

3.1 Framework Web

- **Elegir el enfoque**
 - **Django**: Batteries-included, ORM robusto, admin panel
 - **FastAPI**: Moderno, performance, validación automática
 - **Flask**: Minimalista, control total, curva aprendizaje suave
- **Configuración inicial**
 - Settings/Config por entorno
 - Inicialización de app/proyecto
 - Estructura de módulos

3.2 Manejo de Datos

- **Bases de datos**
 - Modelado de entidades y relaciones
 - Migraciones (Alembic, Django migrations)
 - Indización para performance

- Backups y recuperación
- **ORM o Query Builder**
 - SQLAlchemy, Django ORM, Tortoise ORM
 - Relaciones (one-to-many, many-to-many, etc.)
 - Optimización de queries (N+1 problem)
 - Raw SQL cuando sea necesario
- **Caché**
 - Redis para sesiones y caché de datos
 - Estrategias de invalidación
 - Cache-aside, write-through patterns

3.3 APIs RESTful o GraphQL

- **Endpoints y rutas**
 - Nomenclatura consistente
 - Versionado de API (v1, v2)
 - Documentación automática (OpenAPI/Swagger)
- **Validación de datos**
 - Schemas (Pydantic, Marshmallow)
 - Validación en entrada y salida
 - Mensajes de error claros
- **Autenticación y autorización**
 - JWT, OAuth2, sesiones
 - Roles y permisos
 - Rate limiting
- **Manejo de errores**
 - Excepciones custom
 - Códigos HTTP apropiados
 - Logging de errores para debugging

3.4 Lógica de Negocio

- **Servicios y casos de uso**
 - Separación de lógica de controladores
 - Services/Use cases reutilizables
 - Inyección de dependencias

- **Validaciones complejas**
 - Reglas de negocio en capas adecuadas
 - Transacciones y atomicidad
 - **Procesamiento asíncrono**
 - Task queues (Celery, RQ)
 - Jobs de larga duración
 - Webhooks y callbacks
-

4. 🎨 INTEGRACIÓN DEL FRONTEND

Conectar la interfaz con el backend

- **Servir assets estáticos**
 - Configuración en desarrollo vs producción
 - CDN para optimización
 - **CORS y políticas de seguridad**
 - Configuración de orígenes permitidos
 - Headers de seguridad (CSRF, X-Frame-Options, etc.)
 - **Templates (si aplicable)**
 - Jinja2, Mako u otros
 - Renderizado server-side vs client-side
 - **Integración con frameworks JS**
 - Consumo de APIs desde React, Vue, Angular
 - WebSockets para real-time (si necesario)
-

5. 🧪 TESTING Y CALIDAD

Garantizar que el sistema funciona correctamente

- **Testing unitario**
 - pytest, unittest
 - Mocking y fixtures
 - Cobertura de código
- **Testing de integración**
 - Tests de endpoints

- Base de datos de prueba
 - Fixtures con datos realistas
 - **Testing end-to-end (si aplica)**
 - Selenium, Playwright, Cypress
 - Flujos críticos del usuario
 - **Testing de performance**
 - Load testing (locust, k6)
 - Profiling de código
 - Benchmarks
 - **Análisis de código**
 - Code reviews
 - Deuda técnica
 - Complejidad ciclomática
 - **Seguridad**
 - SAST (Static Application Security Testing)
 - Análisis de dependencias vulnerables
 - Pruebas de penetración
-

6. 📦 DEPLOYMENT E INFRAESTRUCTURA

Llevar a producción

- **Containerización**
 - Docker y Dockerfile
 - Dockercompose para desarrollo
 - Optimización de imágenes
- **Orquestación**
 - Kubernetes o Heroku
 - Declarativos vs imperativos
 - Auto-escalado
- **CI/CD**
 - GitHub Actions, GitLab CI, Jenkins
 - Automated testing en cada push
 - Automated deployments

- **Configuración en producción**
 - Variables de entorno
 - Secretos (AWS Secrets Manager, Vault)
 - Límites de recursos
 - **Monitoreo**
 - Logs centralizados (ELK, Loki)
 - Métricas (Prometheus, Datadog)
 - Alerting
 - Tracing distribuido (Jaeger, Tempo)
 - **Recuperación ante fallos**
 - Health checks
 - Graceful shutdown
 - Circuit breakers
 - Reintentos inteligentes
-

7. 🚀 OPERACIONES Y MANTENIMIENTO

Mantener el sistema en producción

- **Monitoreo continuo**
 - Dashboards de salud del sistema
 - Alertas automáticas
 - Postmortems de incidentes
- **Logging y debugging**
 - Contexto suficiente en logs
 - Trazabilidad de requests (correlation IDs)
 - Debugging en producción (cuando sea seguro)
- **Optimización**
 - Cuellos de botella identificados
 - Profiling en producción
 - Optimización de BD
- **Actualizaciones y parches**
 - Actualización de dependencias
 - Security patches urgentes
 - Feature releases sin downtime

- **Escalado**
 - Crecimiento de base de datos
 - Particionamiento de datos
 - Caching inteligente
 - Separación de servicios (microservicios)
- **Documentación viva**
 - Runbooks para operaciones
 - Arquitectura actualizada
 - Decisiones técnicas (ADRs)

8. 🛠️ HERRAMIENTAS Y ECOSISTEMA

Lo que necesitarás en tu toolbelt

Categoría	Herramientas
Framework	Django, FastAPI, Flask, Tornado
Testing	pytest, coverage, faker, factory_boy
DB	PostgreSQL, SQLAlchemy, Alembic
Cache	Redis, Django cache framework
Task Queue	Celery, RQ, APScheduler
Async	asyncio, aiohttp, ASGI
Admin	Django Admin, FastAPI + custom
Validation	Pydantic, Marshmallow, Cerberus
Logging	Python logging, structlog, Loguru
Monitoring	Prometheus, StatsD, NewRelic
API Docs	OpenAPI/Swagger (automático)
Seguridad	bcrypt, cryptography, python-jose
Email	Django mail, Celery, SendGrid SDK
Search	Elasticsearch, Whoosh, typesense

9. CONCEPTOS TRANSVERSALES

Aplicables a lo largo de todo el proyecto

- **Arquitectura limpia**
 - Separación de capas
 - Inversión de dependencias
 - Testabilidad
- **SOLID principles**
 - Single Responsibility
 - Open/Closed
 - Liskov Substitution
 - Interface Segregation
 - Dependency Inversion
- **Design patterns**
 - MVC, MVT, MVVM
 - Repository, Singleton, Factory
 - Observer, Strategy, Decorator
- **API design**
 - Idempotencia
 - Versionado
 - Paginación
 - Filtrado y búsqueda
- **Seguridad**
 - OWASP Top 10
 - SQL Injection prevention
 - XSS, CSRF
 - Input validation
 - Encryption en tránsito y reposo
- **Performance**
 - Caching strategies
 - Lazy loading
 - Bulk operations
 - Connection pooling

- **Escalabilidad**
 - Stateless design
 - Horizontal scaling
 - Queue-based processing
 - Service independence
-

10. 📖 EVOLUCIÓN: DE JUNIO A SENIOR

Crecimiento profesional

Nivel	Enfoque	Acciones
Junior	Construir features	Completar historias de usuario, escribir tests básicos
Mid	Calidad y arquitectura	Refactoring, mentoring, decisiones de diseño
Senior	Sistemas y estrategia	Escala, performance, decisiones arquitectónicas de largo plazo

✅ Checklist para Validar tu Proyecto

- ☐ Requisitos claramente documentados
 - ☐ Arquitectura dibujada y validada
 - ☐ CI/CD configurado desde el inicio
 - ☐ Tests en cada componente crítico
 - ☐ Logging y monitoreo desde el primer día
 - ☐ Documentación de APIs actualizada
 - ☐ Secrets seguros en todas partes
 - ☐ Plan de escala identificado
 - ☐ Estrategia de deployment definida
 - ☐ Incidentes documentados y post-mortems realizados
-

🎓 Recursos de Aprendizaje

- **Architectures:** "Clean Architecture" (Uncle Bob), "Building Microservices" (Newman)
- **Design Patterns:** "Refactoring Guru", Real Python
- **Security:** OWASP, PortSwigger Web Security Academy
- **Performance:** "High Performance Python" (Gorelick & Ozsvald)

- **DevOps:** "The Phoenix Project", Kubernetes docs
 - **Frameworks:** Official docs (Django, FastAPI)
-

Nota: Este roadmap enfatiza que ser un buen desarrollador Python para web no es solo saber la sintaxis del lenguaje, sino entender el ciclo completo de construcción, deployment y mantenimiento de sistemas.